EUROSIM 2026 Presentation Script

Slide 1 Introduction

Hello everyone. My name is Yu Tao, and today I will present our work on a task-oriented executable modeling framework for smart chemical laboratories.

This work examines how a laboratory task moves from a proposed intent to actual execution. We represent devices, tasks, and operating rules as models and execute them according to defined semantics.

As large language models become involved in laboratory decision-making and control, we also need to define how their requests enter device execution.

Slide 2 Presentation Overview

The presentation has six parts. I will start with the background and the execution boundary problem. Then, I will introduce the unified modeling framework and show how these models work together.

Next, I will use a reactor heating task to show the complete execution sequence. Finally, I will discuss the simulation feedback loop and summarize our work.

Let’s first look at the background.

Slide 3 Background and Execution Boundaries

As large language models advance, smart chemical laboratories are gradually evolving from automated workflow execution toward large language model involvement in decision-making and control.

But whether an execution intent comes from a person, a workflow, or a large language model, we need to explain four things:

How are resources organized and accessed?

How does device state evolve?

How is data generated and transferred?

Does execution stay within allowed limits?

This issue exists even without large language models, but their involvement makes execution boundaries more pressing.

So, we need to carefully consider what kinds of control requests large language models can make and under what conditions these requests can be executed.

Our principle is simple: large language models must not have a privileged bypass. This means that all control requests follow the same execution path.

Therefore, we need an executable framework that keeps resource organization, state evolution, data transfer, and boundary checks aligned.

The next question is how to organize these models.

Slide 4 Unified Modeling Framework

We start with a laboratory task and ask one question: What do we need to model?

First, a task uses devices, so we need to describe their properties and states. We represent this information as a resource model.

Second, the task has steps, an execution order, and data links. We represent these as a workflow model.

Third, execution must follow rules and limits, so we represent these as a constraint model.

We therefore organize the framework around three model types: resources, workflows, and constraints.

The resource layer represents device capabilities and states. The workflow layer organizes steps and data relationships, and the constraint layer defines execution boundaries.

State and data span the layers, while a cross-layer protocol provides shared definitions for control signals, data types, port connections, and execution feedback.

In the diagram, the workflow is on the left, the device resources are on the right, and the constraints are at the bottom.

A workflow node binds to a device capability at runtime. It sends requests to the device model and receives state and data feedback.

Constraints observe the node context and device state and can trigger handling actions when rules are violated.

The framework defines both the objects used to represent a laboratory and the interactions between them.